

บทที่ 3

ทฤษฎีทางด้านการประมวลผลรูปภาพ

กลไกระบบการมองเห็นภาพนั้น จะหมายความรวมถึงทุกสิ่งที่เป็นต่อการได้มา ซึ่งรหัส ทางดิจิทัลที่ใช้แทนภาพนั้นๆ การปรับปรุงเปลี่ยนแปลงข้อมูล และรวมทั้งการนำเสนอภาพที่ได้ หลังการปรับปรุงแล้วด้วย ซึ่งระบบการมองเห็นภาพ (Vision System) นี้อาจมีความยุ่งยากซับซ้อน ต่างกันตามลักษณะการใช้งาน แต่ก็สามารถแบ่งเป็นขั้นตอนกว้าง ๆ ได้ 3 ขั้นตอน ดังนี้

- **การได้มาซึ่งภาพ (Image Acquisition)**

การได้มาซึ่งภาพนี้ จะหมายถึงการแปลงภาพที่เราเห็นในลักษณะทางกายภาพนั้นให้เป็น เซตของข้อมูลทางดิจิทัล โดยในโครงงานนี้จะรับรูปภาพผ่านทางสแกนเนอร์ ทำการแปลงให้เป็น ภาพระดับสีเทา และเก็บค่าความสว่างของแต่ละพิกเซลอยู่ในรูปแบบเลขฐานสองจำนวน 8 บิต ซึ่งเซตของพิกเซลเหล่านี้จะถูกส่งไปยังส่วนประมวลผลภาพต่อไป

- **กระบวนการประมวลผลภาพ (Image Processing)**

หน้าที่พื้นฐานของกระบวนการประมวลผลภาพ ได้แก่ การสร้างภาพใหม่โดยแยกแยะส่วน ที่เราต้องการหรือสนใจออกจากสิ่งรบกวนออกจากกัน (Noise Elimination), การหาขอบภาพ (Edge Enhancement), การกรอง (Filtering), การปรับปรุงเปลี่ยนแปลงค่าระดับสีเทา (Grey scale Modification), การประมวลผลภาพเชิงตัวเลข (Digital Image Processing) หมายถึง การนำภาพมาประมวลผลด้วยเครื่องคอมพิวเตอร์ โดยภาพที่นำมาประมวลผลนี้จะถูกแทนที่ด้วยตัวเลขให้อยู่ในรูปแบบของเมตริกซ์ แต่ภาพที่ได้โดยส่วนมากแล้วจะเป็นภาพที่ได้จากตัวรับสัญญาณ ซึ่งอยู่ในรูปของฟังก์ชันต่อเนื่อง $f(x, y)$ ในระนาบสองมิติ (คือแกน x และแกน y) โดยจะเป็นสัดส่วนกับความ สว่างหรือความเข้มของภาพ ที่ตำแหน่ง (x, y) ซึ่งเรียกว่า ระดับสีเทา (Grey Level)

- **ผลที่ได้และการแสดงผล (Output and Display)**

รูปแบบของการแสดงผลที่ได้นั้นขึ้นอยู่กับว่าเราจะนำข้อมูลที่ได้ไปใช้ต่อในลักษณะใด โดยอาจแสดงผลทางเครื่องพิมพ์ หรือแสดงผลผ่านทางหน้าจอ เป็นต้น

การประมวลผลรูปภาพสามารถทำได้ในสองลักษณะใหญ่ๆคือ การปรับปรุงคุณภาพของภาพให้ดีขึ้นเพื่อให้มนุษย์ดู และกระบวนการ (process) การนำข้อมูลรูปภาพเพื่อนำไปใช้ต่อไปในการเก็บ, การส่งข้อมูล และการนำข้อมูลรูปภาพมาแสดงให้มนุษย์เห็นบนเครื่องกลใด ๆ

3.1 การแทนภาพด้วยข้อมูลแบบดิจิทัล

ภาพข้อมูลแบบดิจิทัล (Digital Image) เป็นภาพที่ถูกแปลงมาจากภาพอนาล็อกให้อยู่ในรูปของตัวเลข โดยการนำภาพอนาล็อกมาแบ่งเป็นพื้นที่สี่เหลี่ยมเล็ก ๆ ที่เรียกว่า พิกเซล (pixel) ในแต่ละพิกเซลจะถูกระบุตำแหน่งโดย (x, y) และค่าระดับสีเทาของพิกเซล โดยเราสามารถแปลงภาพเป็นข้อมูลแบบดิจิทัลได้ โดยมีขั้นตอนและวิธีการดังนี้

เมื่อเรานำสัญญาณอนาล็อกที่ต้องการประมวลผลมาผ่านส่วนที่เรียกว่า ดิจิไทเซอร์ (Digitizer) ซึ่งจะมีหน้าที่ในการเปลี่ยนสัญญาณอนาล็อกให้เป็นสัญญาณดิจิทัล จากนั้นทำการควอนไทซ์ (Quantizing) เพื่อที่จะประมวลสัญญาณด้วยระบบคอมพิวเตอร์ ฟังก์ชันของภาพ $f(x,y)$ จะถูกทำให้เป็นสัญญาณไม่ต่อเนื่องทั้งระนาบของภาพ ซึ่งเราเรียกว่าการสุ่มภาพ (Image Sampling) ของฟังก์ชันที่ได้เรียกว่า การควอนไทซ์ระดับสีเทา (Grey level quantization) ก็จะได้ข้อมูลที่เป็นดิจิทัล

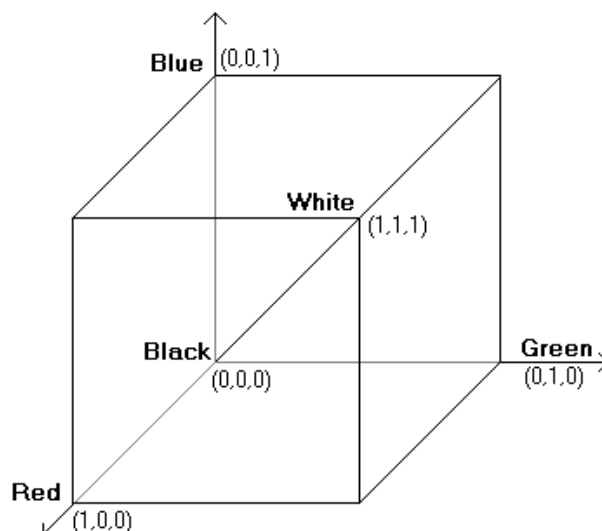
3.2 การประมวลผลแบบสเปเชียล (Spatial Domain Processing)

ในการประมวลผลแบบสเปเชียลเราจะแบ่งรูปภาพออกเป็นฟังก์ชันสองตัวแปร $f(x,y)$ ซึ่ง x และ y คือโคออร์ดิเนต (Coordinate) ของภาพนั้นๆ และขนาดของ f ณ ทุกๆโคออร์ดิเนต (x,y) คือความเข้มสี (Intensity) ของรูปภาพ ณ จุดนั้นๆ ทุกๆ x,y และ f มีขนาดที่จำกัดและเป็นฟังก์ชันต่อเนื่อง (Discrete Function) เราจะเรียกรูปภาพนั้นๆว่ารูปภาพดิจิทัล ขอบเขตของการประมวลผลรูปภาพทางดิจิทัลคือการประมวลผลโดยใช้ดิจิทัลคอมพิวเตอร์

3.3 ทฤษฎีสี RGB, YCbCr และการเปลี่ยนรูป

3.3.1 สี RGB

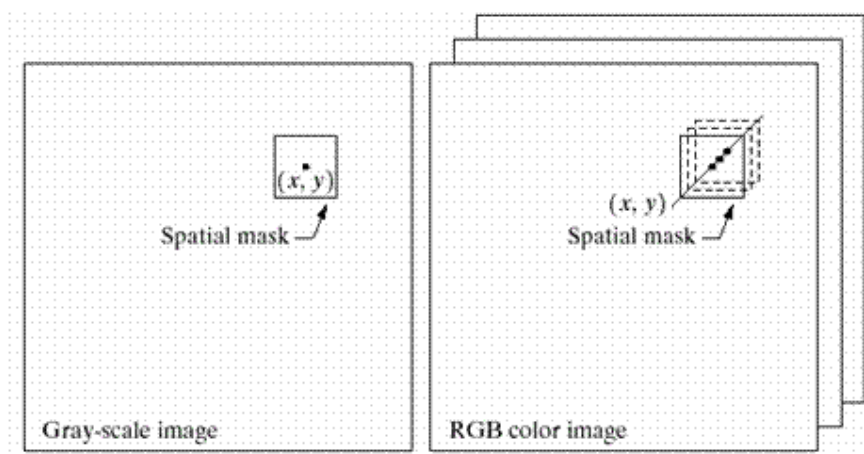
เป็นสีที่อยู่บนโคออร์ดิเนตของคาร์ทีเซียน (Cartesian Coordinate System) มีแม่สีหลักอยู่ด้วยกันสามสีคือ แดง เขียว และน้ำเงิน



รูปที่ 3-1 โคออร์ดิเนตคาร์ทีเซียนของสี RGB

รูปภาพที่แสดงด้วย RGB จะประกอบด้วยรูปเกรย์สเกล (Grey scale) ทั้งหมดสามเลเยอร์ (Layer) เข้าด้วยกัน คือเลเยอร์ของสีแดง เลเยอร์ของสีเขียว และเลเยอร์ของสีน้ำเงินเข้ามารวมกันเป็นหนึ่งเลเยอร์ และสามารถแสดงทั้งสีแดง เขียว และน้ำเงินได้

$f(x,y)$ ในโหมด RGB ณ จุดใดๆ = {R,G,B}



รูปที่ 3-2 เปรียบเทียบเลเยอร์ระหว่างรูปเกรย์สเกลและรูปภาพ RGB

จำนวนของสีที่สามารถแสดงได้ จะแปรตามจำนวนของบิตที่ใช้แสดงค่าความเข้มของแต่ละสีซึ่งเรียกว่า Pixel Depth รูปภาพที่มีจำนวน pixel depth 8 บิตหมายถึงรูปภาพที่มีจำนวนบิตของแต่ละสีเท่ากับ 8 บิต รวมเข้าด้วยกันสามสี ตามรูปที่ 3-2 จะกลายเป็นภาพขนาด 24 บิต สามารถแสดงจำนวนสีได้ $= (2^8)^3 = 16,777,216$ สี จะมีค่าของความเข้มของแต่ละสีของทั้ง 3 สี เป็น 256 สี แสดงได้ระหว่าง 0-255 ดังตาราง

Number System	Colour Equivalents					
Decimal	0	51	102	153	204	255
Hex	00	33	66	99	CC	FF

ตารางที่ 3-1 ตัวอย่างค่าของสีต่างๆใน RGB

3.3.2 ระดับสีเทา (Grey Scale)

หากเราต้องการค่าข้อมูลที่ละเอียดมากขึ้น ก็จำเป็นที่จะต้องเพิ่มจำนวนบิตการแสดงผลค่าของแต่ละพิกเซล ยกตัวอย่างเช่น หากแบ่งความเข้มของการส่องสว่างให้มี 4 ระดับ ก็ต้องใช้เลขฐานสองจำนวน 2 บิต และจำนวน 4 บิตสำหรับ 16 ระดับ, 8 บิตสำหรับ 256 ระดับ ซึ่งจำนวนระดับที่ใช้ในระดับสีเทาหรือระดับเกรย์นี้มักเป็นเลขยกกำลังของ 2, ค่าที่ต่ำที่สุดคือ 0 กำหนดให้เป็นสีดำ และ 1 หรือตัวเลขที่น้อยกว่าค่าสูงสุดของระดับเกรย์อยู่ 1 (เช่น 15 สำหรับระดับเกรย์ 16 ระดับ) แทนสีขาว ค่าที่กำหนดให้ในแต่ละ

พิกเซลนี้นิยมใช้เป็นจำนวนเต็ม ในยุคแรก ๆ ของระบบการมองเห็นภาพจะใช้ระบบเลขฐานสอง แต่ในปัจจุบันเทคโนโลยีไมโครโปรเซสเซอร์เข้ามามีบทบาทมากขึ้น การแบ่งระดับเป็น 16, 64 หรือ 256 เป็นเรื่องธรรมดา แต่ทั้งนี้ในการมองเห็นของมนุษย์จะสามารถแยกแยะความแตกต่างได้เพียง 10-15 ระดับเท่านั้น การแบ่งโดยละเอียดเป็น 64 หรือ 256 ระดับอาจนำไปประยุกต์ใช้กับงานการประมวลผลภาพหรือกระบวนการทางอุตสาหกรรมอื่นๆ ซึ่งจะเห็นว่าจำนวนระดับเกรย์จะเป็นตัวจำกัดรายละเอียดของภาพโดยทั่วไปแล้วยังแบ่งระดับเกรย์เป็นหลายระดับก็จะเป็นการเพิ่มคุณภาพของภาพด้วย และการเพิ่มจำนวนพิกเซล เช่น จาก $35 * 35$ เป็น $250 * 250$ ก็จะเป็นการเพิ่มความละเอียด (Resolution) และรายละเอียด (Detail) ของภาพเช่นกัน จะเห็นว่าจะแตกต่างกับการขยายภาพ (Zoom) โดยการขยายภาพนี้เป็นการขยายแต่ละพิกเซลให้ใหญ่ขึ้น ไม่ได้เป็นการเพิ่มจำนวนพิกเซล

3.3.3 สี YCbCr

เป็นแม่สีที่นิยมใช้กันอย่างแพร่หลายในกล้องถ่ายรูป กล้องถ่ายภาพวิดีโอ และกล้องวงจรปิดต่างๆ ที่บันทึกในรูปแบบดิจิทัล โดยจะแบ่งออกเป็นแม่สีสามตัวเหมือนกับ RGB แม่สีแต่ละตัวจะแสดงข้อมูลของในพิกเซลนั้นๆ คือ

- แม่สีที่กำหนดความเข้มของแสง
- แม่สีที่กำหนดความเข้มของสี

แม่สีที่เป็นตัวกำหนดความเข้มของแสงในพิกเซลนั้นๆ โดยเฉพาะ ซึ่ง

Y จะเป็นค่าความเข้มแสง (Luminance)

Cb แสดงถึงส่วนของโทนสีน้ำเงินโดยไม่มีความเข้มแสงมาเกี่ยวข้อง

Cr แสดงถึงส่วนของโทนสีแดงโดยไม่มีความเข้มแสงมาเกี่ยวข้อง

3.3.4 การเปลี่ยนรูประหว่าง RGB และ YCbCr

การแปลงรูประหว่าง RGB และ YCbCr สามารถแปลงโดยใช้หลักการของเมทริกซ์และจะได้สมการดังต่อไปนี้

$f(x,y)$ ในโหมด RGB ณ จุดใดๆ = $\{R,G,B\}$

$f(x,y)$ ในโหมด YCbCr ณ จุดใดๆ = $\{Y,Cb,Cr\}$

$$Y = 0.299 R + 0.587 G + 0.114 B$$

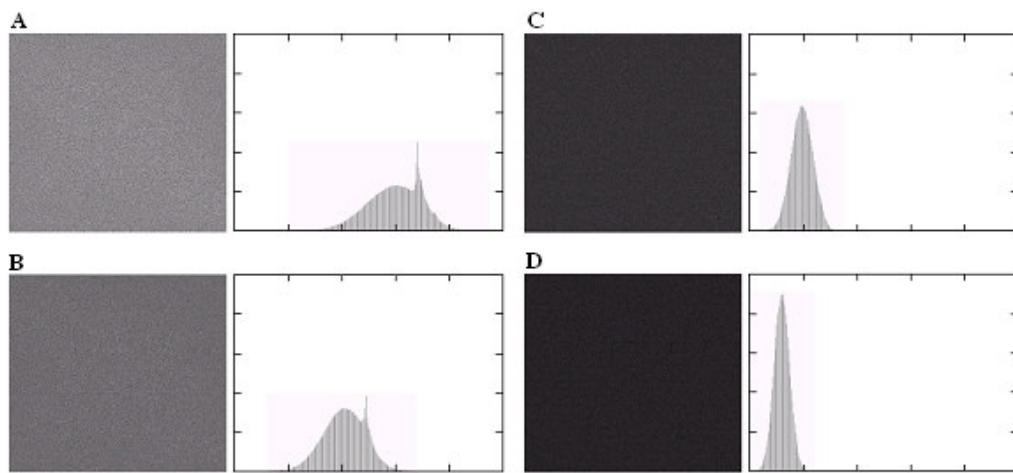
$$Cb = -0.1687 R - 0.3313 G + 0.5 B + 128$$

$$Cr = 0.5 R - 0.4187 G - 0.0813 B + 128$$

3.4 ฮิสโตแกรม (Histogram)

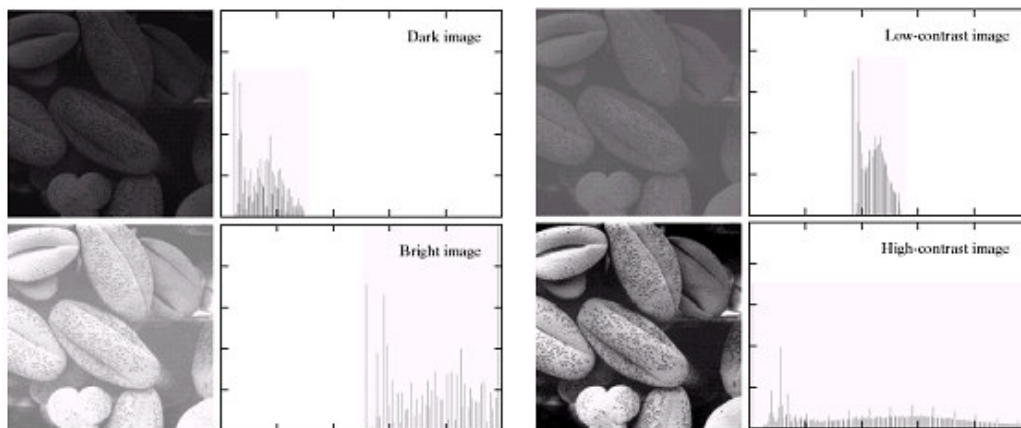
ฮิสโตแกรมของรูปภาพคือกลุ่มของผลรวมของความเข้มสีของพิกเซลทั้งหมดในรูป มีค่าตั้งแต่ $[0, \text{ขนาดภาพ}-1]$ ในฟังก์ชันต่อเนื่อง $h(r_k) = n_k$ ซึ่ง r_k เท่ากับความเข้มแสง k หน่วยในภาพ n_k เท่ากับจำนวนพิกเซลทั้งหมดในภาพที่มีความเข้มแสงเท่ากับ r_k รูปภาพที่มีการกระจายของฮิสโตแกรมที่สม่ำเสมอจะแสดงรายละเอียดได้ชัดกว่ารูปภาพที่ฮิสโตแกรมแคบและกระจุกอยู่ใกล้ๆ กัน

ฮิสโตแกรมเป็นเทคนิคพื้นฐานที่สุดของการประมวลผลรูปภาพในสเปซเชิงโดเมน ฮิสโตแกรมมีประโยชน์มากทางด้านการปรับรูปภาพที่มีแสงมากหรือน้อยเกินไปและดูด้วยตาเปล่าไม่ชัดให้อยู่ในระดับที่สามารถดูด้วยตาเปล่าได้ชัดเจน อีกทั้งยังสามารถใช้เป็นเทคนิคร่วมกับการบีบอัดรูป และการตัดแบ่งส่วนรูป (Image Compression and Image Segmentation)



รูปที่ 3-3 รูปภาพที่มีช่วงของฮิสโตแกรมต่างกัน

รูปภาพที่มีฮิสโตแกรมกระจายอย่างสม่ำเสมอจะส่งผลให้คอนทราสต์ (Contrast) หรือความชัดเจนสูง ในขณะที่รูปภาพที่ฮิสโตแกรมไม่กระจายอย่างสม่ำเสมอ แต่อยู่ที่ด้านใดด้านหนึ่งมากเป็นพิเศษจะเรียกว่ารูปภาพที่มีคอนทราสต์ต่ำ ซึ่งมองด้วยตาเปล่าแล้วจะแยกระายละเอียดของภาพได้ยากกว่าภาพที่มีคอนทราสต์สูง



รูปที่ 3-4 ความสัมพันธ์ระหว่างฮิสโตแกรมและคอนทราสต์

3.4.1 การปรับฮิสโตแกรมของรูปภาพ

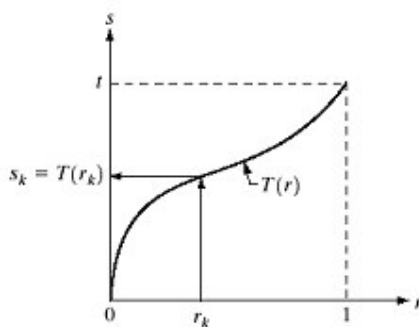
เมื่อเราทราบความสัมพันธ์ระหว่างฮิสโตแกรมและคอนทราสต์แล้ว การจะทำให้รูปภาพมีความคมชัดขึ้นจึงเป็นเรื่องที่สามารถทำได้ด้วยฮิสโตแกรม ทำได้ด้วยการบีบอัดส่วนของกลุ่มสีให้กระจายออกอย่างสม่ำเสมอให้เต็มรูปภาพ

$$s = T(r) \quad ; 0 \leq r \leq 1$$

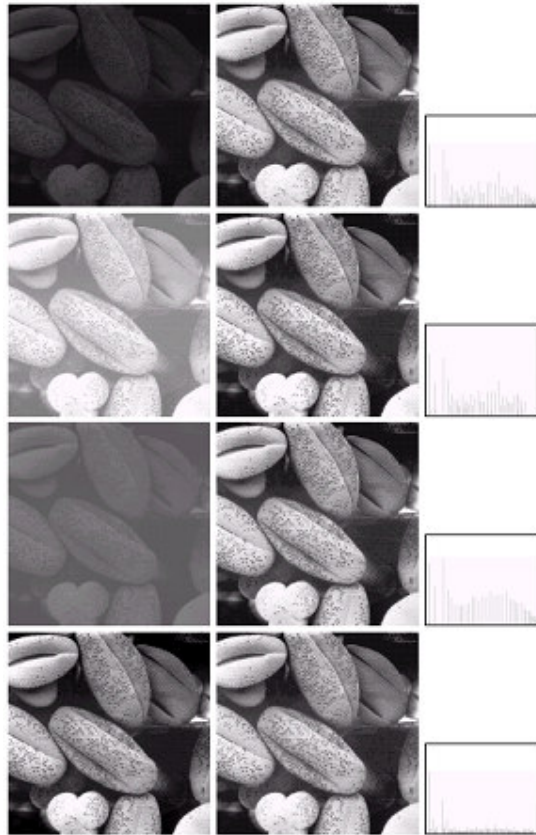
เมื่อ r = จำนวนความเข้มสีทั้งหมดที่แสดงได้ในรูปด้นฉบับ อยู่ในเรนจ์ $[0,1]$

- พิกเซลนั้นจะเป็นสีดำเมื่อ $r=0$
- พิกเซลนั้นจะเป็นสีขาวเมื่อ $r=1$

$T(r)$ = จำนวนพิกเซลทั้งหมดของรูปภาพที่มีค่าความเข้มแสงเท่ากับ r



รูปที่ 3-5 ความสัมพันธ์ระหว่าง S และ $T(r)$



รูปที่ 3-6 รูปต้นฉบับที่มีฮิสโตแกรมต่างกันที่ความเข้มแสงจะถ่าย เมื่อนำมาปรับฮิสโตแกรมแล้ว

จากรูป 3-6 จะเห็นได้ว่ารูปต้นฉบับที่ถูกถ่ายด้วยสภาพแวดล้อมและความเข้มแสงที่ต่างกัน เมื่อนำมาทำการปรับฮิสโตแกรมให้กระจายอย่างสม่ำเสมอแล้ว จะมีคอนทราสต์ หรือความคมชัดที่ดูได้ด้วยตาเปล่า มากขึ้น เทคนิคการปรับฮิสโตแกรมนี้จึงเป็นเทคนิคที่นิยมใช้กันอย่างแพร่หลาย

3.5 การนอร์มัลไลเซชันของสี RGB (Normalization)

ค่านอร์มัลไลซ์ของสี RGB เป็นการแสดงอัตราส่วนและความคมชัดของสีแต่ละสี

$f(x,y)$ ณ จุดใดๆ = {R,G,B}

$f(x,y)$ นอร์มัลไลซ์ = {r,g,b} ซึ่ง

$$r = \frac{R}{R+G+B}$$

$$g = \frac{G}{R+G+B}$$

$$b = \frac{B}{R+G+B}$$

3.6 การทำเทรชโฮล (Threshold)

อุปกรณ์ที่มีความสามารถในการแสดงผลได้แค่ 2 ระดับ หรือ 2 สี คือ สีขาวกับสีดำยังมีการใช้กันอย่างแพร่หลาย เช่น เครื่องพิมพ์ (Printer), เครื่องโทรสาร (Fax), จอภาพแสดงผลแบบโมโนโครม (Monochrome Monitor) เป็นต้น เนื่องจากอุปกรณ์เหล่านี้เป็นอุปกรณ์ที่มีราคาถูก ดังนั้นจึงเกิดปัญหาในการแสดงผลขึ้น การที่จะแสดงผลหรือพิมพ์รูปภาพที่มีระดับความเข้มของภาพหลายระดับซึ่งมีมากกว่าความสามารถในการแสดงผลของอุปกรณ์เหล่านั้นที่มีเพียงแค่ 2 ระดับ เราจะต้องทำการแปลงข้อมูลภาพให้เป็นภาพไบนารี (Binary Image) เสียก่อน ซึ่งการสร้างภาพไบนารีนั้นก็หมายถึงการแปลงข้อมูลภาพที่มีระดับความเข้มหลายระดับ (Multilevel Image) ให้เป็นภาพที่มีระดับความเข้มเพียง 2 ระดับ นั่นคือ 1 จุดภาพ มีได้ 2 ค่าเท่านั้น คือ 0 กับ 1 โดยจุดภาพที่แทนด้วย 0 หมายถึงจุดภาพที่มีสีดำ ส่วนจุดที่แทนด้วยค่า 1 จะหมายถึงจุดภาพที่มีสีขาว เมื่อทำการแปลงเป็นภาพไบนารีแล้ว จึงสามารถนำภาพนั้นไปแสดงผลบนอุปกรณ์เหล่านั้น จะเห็นได้ว่าการแปลงข้อมูลภาพหลายระดับเป็นภาพไบนารีจึงมีความจำเป็นและมีประโยชน์มากในการแสดงผลภาพที่มีความเข้มของภาพหลายระดับบนอุปกรณ์ที่มีความสามารถในการแสดงผลได้ 2 ระดับ สำหรับประโยชน์อีกประการหนึ่งในการแปลงข้อมูลภาพนั้นเป็นภาพไบนารี คือการลดเนื้อที่การเก็บข้อมูลภาพ ภาพไบนารีจะสามารถลดเนื้อที่ลงได้ถึง 8 เท่า นั่นคือถ้าเดิม 1 จุดภาพใช้เนื้อที่ในการเก็บ 8 บิต เมื่อทำการแปลงเป็นภาพไบนารีแล้วจะใช้เนื้อที่ในการเก็บ 1 จุดภาพนั้นเพียง 1 บิตเท่านั้น อีกทั้งยังสามารถนำไปประยุกต์ใช้งานได้อย่างแพร่หลาย เช่น นำไปประยุกต์ใช้ในการวิเคราะห์เอกสารในขั้นตอนที่เรียกว่าการประมวลผลขั้นต้น (Preprocessing) เป็นต้น

ในการสร้างภาพไบนารี สามารถทำได้โดยใช้เทคนิคการทำเทรชโฮล (Thresholding Technique) โดยพิจารณาว่าจุดภาพใดควรจะเป็นจุดขาวหรือจุดดำ จะกระทำโดยการเปรียบเทียบระหว่างจุดภาพเริ่มต้นกับค่าคงที่ค่าหนึ่งซึ่งเรียกว่า ค่าเทรชโฮล (Threshold Value) เทคนิคนี้ใช้กันมากในกรณีที่มีข้อมูลภาพมีลักษณะแตกต่างกันระหว่างวัตถุ (Object) และพื้นหลัง (Background) โดยค่าของจุดภาพใด ๆ ที่มีค่าน้อยกว่าค่าเทรชโฮลจะถูกกำหนดให้เป็น 0 (จุดดำ) และถ้าค่าของจุดภาพใด ๆ ที่มีค่ามากกว่าหรือเท่ากับค่าเทรชโฮล จะถูกเปลี่ยนให้เป็น 1 (จุดขาว)

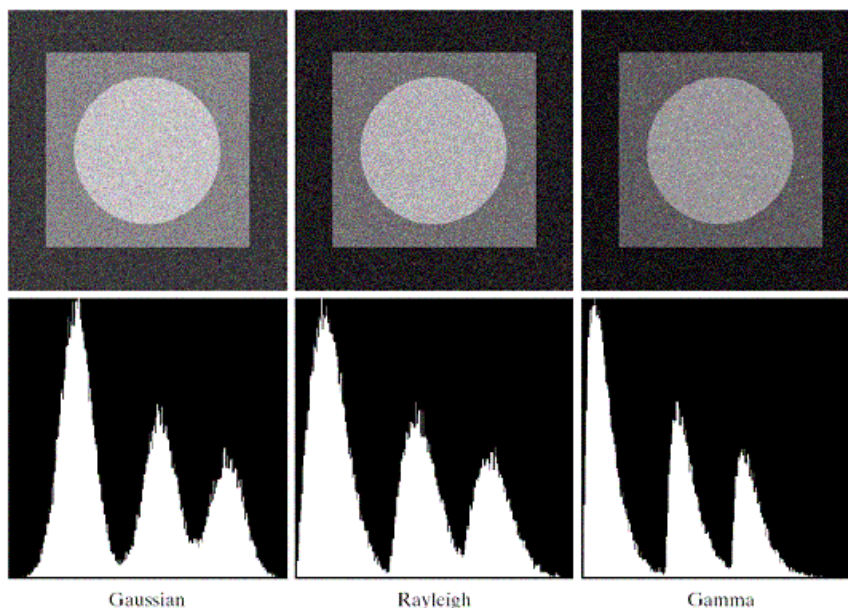
ในการสร้างภาพไบนารีโดยใช้เทคนิคเทรชโฮลเพื่อให้ได้ผลลัพธ์ที่เหมาะสมและคมชัด สิ่งที่สำคัญที่สุดคือ ค่าเทรชโฮล เนื่องจากถ้าเลือกค่าเทรชโฮลที่ไม่เหมาะสม (ค่าเทรชโฮลที่มีค่าน้อยเกินไปหรือมากเกินไป) ภาพที่ได้อาจจะไม่เหมาะสม ขาดความคมชัด และอาจทำให้รายละเอียดบางส่วนขาดหายไป กล่าวคือภาพที่ได้อาจจะมืดเกินไปหรือสว่างเกินไป หรือภาพที่ได้มีสิ่งรบกวน (Noise) เกิดขึ้น อันเป็นผลทำให้ภาพผลลัพธ์ที่ได้ไม่สวยงามเท่าที่ควร ดังนั้นปัญหาของการสร้างภาพไบนารีโดยวิธีเทรชโฮลนี้คือ ทำอย่างไรจึงจะสามารถคำนวณหาค่าเทรชโฮลที่เหมาะสมสำหรับแต่ละภาพที่จะนำมาทำการสร้างภาพไบนารี ซึ่งมีวิธีคำนวณหาค่าเทรชโฮลหลายวิธี โดยแต่ละวิธีเหมาะสมกับลักษณะการทำงานที่แตกต่างกันไป เช่น

- การหาค่าเทรชโฮลโดยการกำหนดค่าล่วงหน้า (Pre-assigned Threshold Value)
- การหาค่าเทรชโฮลจากค่ากลาง (Mid-range Threshold Value)

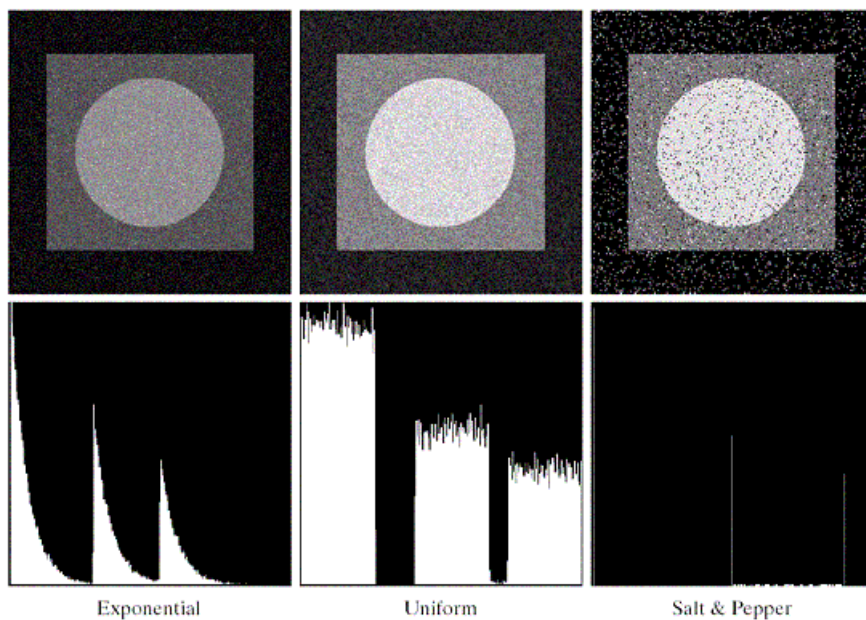
3.7 ตัวรบกวนภาพชนิดต่างๆและการกำจัด (Noise Model and Noise Reduction)

เมื่อภาพที่ถ่ายถูกรบกวนด้วยสิ่งแวดล้อมไม่ว่าจะในขณะที่ถ่ายภาพ หรือขณะที่ส่งข้อมูลภาพไปที่ต่างๆ เช่นภาพถ่ายจากดาวเทียมที่ส่งกลับมายังโลก จะรบกวนให้ภาพผิดเพี้ยนไปจากภาพต้นฉบับ มีการจำแนกประเภทตัวรบกวนต่างๆออกเป็น 6 ชนิดใหญ่ๆดังนี้

- **เกาเซียน (Gaussian Noise)**
เกิดจากการรบกวนของวงจรอิเล็กทรอนิกส์ ทำให้ภาพที่ถ่ายได้เกิดการผิดเพี้ยนไป
- **รีเลย์ (Rayleigh Noise)**
พบมากจากภาพถ่ายที่ต้องถ่ายระยะทางไกล เช่นภาพถ่ายทางอากาศ
- **แกมมา (Gamma Noise)**
พบมากจากภาพที่ถ่ายด้วยเลเซอร์
- **เอกซ์โปเนนเชียล (Exponential Noise)**
พบมากจากภาพที่ถ่ายด้วยเลเซอร์
- **ยูนิฟอร์ม (Uniform Noise)**
เป็นตัวรบกวนที่มีการกระจายตัวอย่างเท่าเทียมกัน ไม่ค่อยเป็นที่พบเห็นนัก
- **ซอลท์แอนด์เพปเปอร์ (Salt and Pepper Noise)**
เกิดจากการสปีคของเซนเซอร์ทำให้เกิดจุดสีขาวและสีดำขึ้นทั่วทั้งรูป



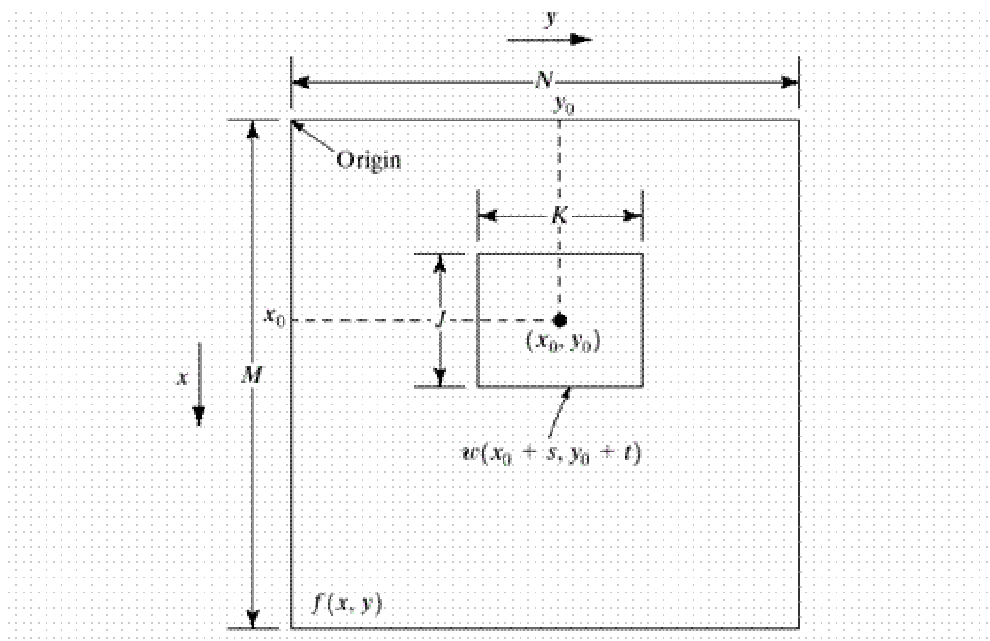
รูปที่ 3-7 ตัวรบกวนภาพชนิดต่างๆ



รูปที่ 3-7 ตัวรบกวนภาพชนิดต่างๆ (ต่อ)

3.7.1 การทำฟิลเตอร์เพื่อกำจัดตัวรบกวนภาพ

การทำฟิลเตอร์ทำได้โดยจะใช้วินโดว์ขนาดเล็กๆ ที่สร้างขึ้นมา โดยที่ค่าในแต่ละพิกเซลของวินโดว์จะเป็นตัวกำหนดพิกเซลของรูปที่ผ่านการฟิลเตอร์แล้ว โดยพิกเซลรอบข้างของภาพต้นแบบจะมีผลต่อพิกเซลใหม่ในภาพที่ผ่านการฟิลเตอร์แล้ว



รูปที่ 3-8 การทำฟิลเตอร์ภาพขนาด (x,y) ด้วยวินโดว์ขนาด (j,k)

3.7.1.1 ฟิลเตอร์ค่ามัธยฐาน (Median Filtering)

การคำนวณค่ามัธยฐานทำได้โดยนำค่าทั้งหมดในวินโดว์มาเรียงลำดับจากมากไปน้อย และนำค่ากลาง ซึ่งก็คือค่ามัธยฐานมาเป็นพิกเซลที่ผ่านการกรองแล้ว ดังสมการ

$$g(x,y) = \text{med} (f(x+i,y+j))$$

$$i = -a \text{ to } a$$

$$j = -b \text{ to } b$$

a และ b คือจำนวนพิกเซลที่นำมาหาค่าเฉลี่ย ปกติแล้วจะเท่ากับความกว้างและความสูงของวินโดว์

3.8 การขยายขอบเขตของภาพ (Region Growing)

การขยายขอบเขตของภาพเป็นการขยายส่วนย่อยๆ ในภาพให้เป็นกลุ่มใหญ่ๆ ขึ้น โดยนำกลุ่มของพิกเซลย่อยๆ ที่มีคุณลักษณะคล้ายกันนำมารวม ตามข้อกำหนดที่ได้กำหนดไว้ล่วงหน้า

3.9 การแปลงภาพทางเรขาคณิต (Geometric Transformation)

ในการเปลี่ยนขนาดหรือมุมมองของภาพที่มีอยู่ให้มีขนาดใหญ่ขึ้น หรือหมุนให้มีมุมมองที่พอเหมาะต่อการนำไปประมวลผล สามารถใช้ความรู้ทางเรขาคณิตโดยใช้สมการในรูปแบบต่อไปนี้

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_0 & a_1 \\ b_0 & b_1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a_2 \\ b_2 \end{bmatrix}$$

3.9.1 การปรับขนาดของภาพ (Image Resizing)

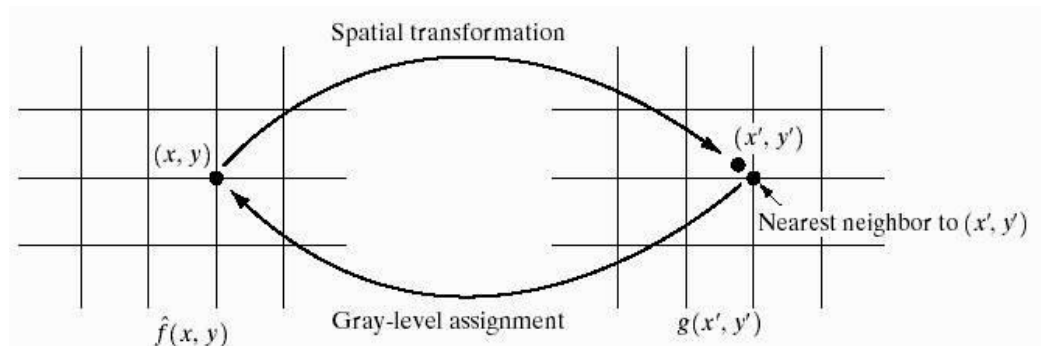
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

3.9.2 การหมุนภาพ (Image Rotation)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

3.9.3 ปัญหาของ Forward Mapping และ Backward Mapping

ในการแปลงตำแหน่งจากจุดเดิม (x, y) ไปเป็นจุดใหม่ (x', y') โดยวิธีทางเรขาคณิตนั้นถ้าเราแปลงโดยใช้วิธีคำนวณโดยเริ่มต้นจากจุดเดิมแปลงไปเป็นจุดใหม่หรือการแปลงภาพไปด้านหน้า (forward mapping)



รูปที่ 3-9 การแปลงภาพไปด้านหน้า และกลับหลัง

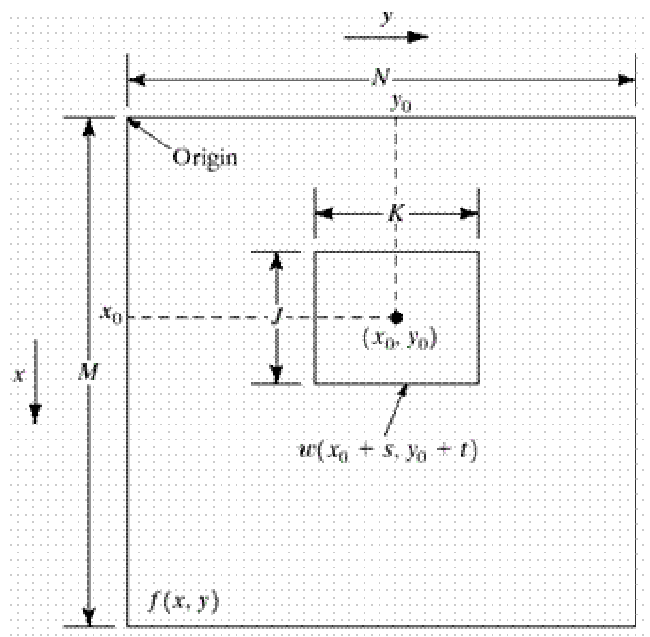
ดังนั้นผลที่ได้ในจุดใหม่ (x', y') อาจทำให้มีบางจุด x', y' ที่ไม่ได้รับการแปลงมา ทำให้ภาพขาดหายไป รวมทั้งอาจมีบางจุดที่ แปลงไปซ้ำสองครั้ง ทำให้ภาพที่ได้ออกมาผิดเพี้ยนจากที่ควรเป็นมีแนวทางแก้ไข คือ ใช้ จุดใหม่แล้วแปลงกลับไปหาจุดของภาพต้นแบบ หรือการแปลงภาพกลับหลัง(backward mapping) โดยใช้สมการกลับกับสมการเดิม

3.10 การทำแมตชิ่งโดยวิธีการคอรีเลชัน (Template matching using Correlation)

วิธีการนี้จะใช้ภาพต้นแบบขนาดเล็ก $w(x,y)$ ขนาด $J \times K$ ภายใต้ภาพใหญ่ $f(x,y)$ ซึ่งมีขนาด $M \times N$ โดยที่ $J \leq M$ และ $K \leq N$ โดยสามารถอธิบายโดยใช้สมการคอรีเลชัน การหาค่าคอรีเลชันแบบที่ง่ายที่สุด ระหว่างค่า $f(x,y)$ กับ $w(x,y)$ คือ

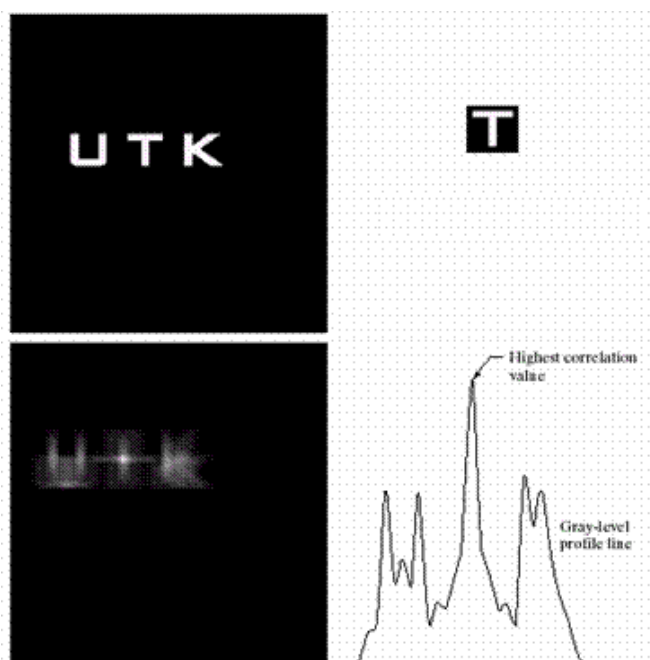
$$c(x,y) = \sum \sum f(s,t) \cdot w(x+s,y+t)$$

โดย $x = \{0,1,2,\dots,M-1\}$ และ $y = \{0,1,2,\dots,N-1\}$ โดยค่าผลรวมในสมการเป็นค่าผลรวม ณ ตำแหน่งที่ภาพ w กับ f ซ้อนทับกัน



รูปที่ 3-10 แสดงตำแหน่งของวินโดวในการคอร์เรเลตแมตซิง

จากรูป สมมติให้ตำแหน่งจุดออริจิน(origin) ของภาพ $f(x)$ อยู่ทีบนซ้ายและ ของภาพ $w(x)$ อยู่ตรงกลางภาพ ค่า χ ตำแหน่งใกล้เคียงที่สุดจะได้ค่าสูงที่สุด โดย จากภาพ χ ตำแหน่งด้านบนของตัว T จะได้ค่าคอร์เรลชัน สูงที่สุด แต่วิธีการนี้จะมีปัญหาคือใช้เวลามากเนื่องจากจะต้องนำภาพเล็กแทนกับภาพใหญ่ทั้งภาพ และมีปัญหาในเรื่องที่ลักษณะของรูปแบบที่ต้องการจะจดจำไม่ควรจะเปลี่ยนแปลงไปมากนักอาจจะทำให้จดจำรูปแบบผิดได้

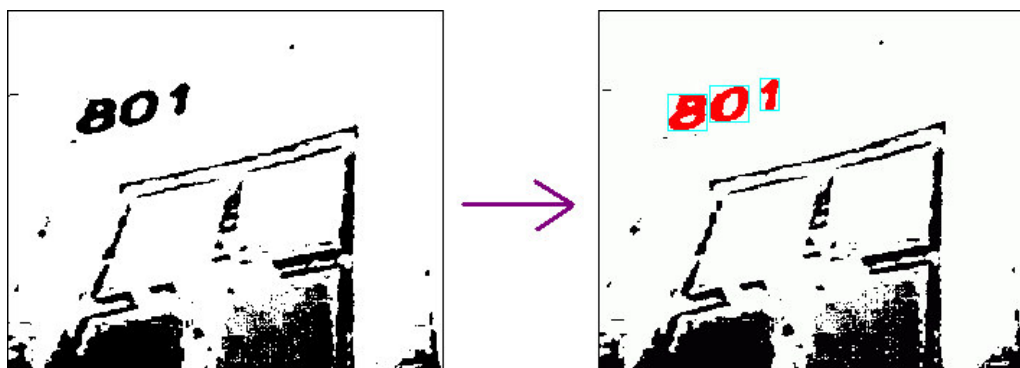


รูปที่ 3-11 ผลของการคอร์เรเลตแมตซิง

จากผลที่ได้จะเห็นว่าตำแหน่งสูงที่สุดเมื่อนำวินโดว์ T ไปวิ่งในรูปจะเป็นตำแหน่งที่ตัวอักษร T อยู่ และจะเห็นได้อย่างค่อนข้างชัดเจน แต่การทำคอรีเลชันนี้หากตัวอักษรเปลี่ยนแปลงไปเช่นมีขนาดที่ไม่เท่าเดิม มุมองศาที่เปลี่ยนไป ก็จะทำให้ผลการทำคอรีเลชันนี้ผิดเพี้ยนไปได้อย่างง่ายดาย

3.11 หลักการจดจำรูปแบบโดยใช้วิธีสโตรป (Pattern Recognition by Strobes)

หลักการสโตรปคือเป็นการจดจำรูปแบบโดยใช้การนับค่าเฉลี่ยของระดับการเปลี่ยนแปลงของสีในแต่ละแถว โดยถ้าต้องการให้ละเอียดมากขึ้นก็สามารถทำการแบ่งส่วนออกมาเพื่อดูจุดสนใจในแต่ละส่วนได้ ในขั้นแรกจะต้องมีโดเมนของส่วนที่นำมาทำสโตรป



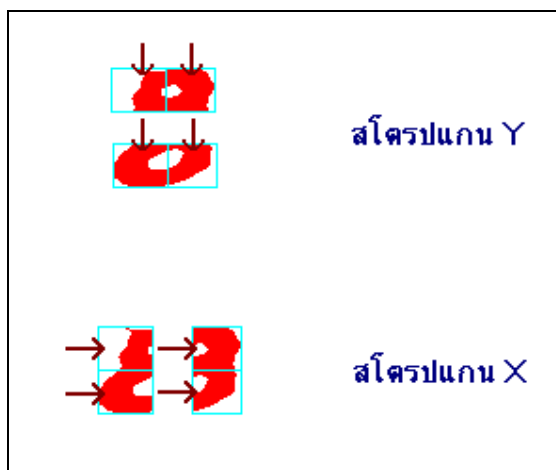
รูปที่ 3-12 การแบ่งหลักของตัวเลข

และนำตัวอักษรที่จะนำมาตรวจสอบสโตรปมาตัดแบ่งออกเป็นช่องในแนวดิ่งและแนวนอน เท่าๆกัน ทั้งนี้การทำสโตรปไม่มีจำนวนของช่องที่ตายตัว แต่การแบ่งเป็นช่องจำนวนมากเท่าใดก็就会有ความถูกต้องมากขึ้นเท่านั้น แต่ก็จะมีจำนวนช่องของสโตรปที่ต้องวิเคราะห์มากขึ้นด้วย จำนวนช่องของสโตรปมีได้ตั้งแต่ 1×1 , 2×2 , 3×3 , 4×4 , ... เป็นต้นไป ในตัวอย่างนี้จะเป็นการทำสโตรปด้วยช่องขนาด 2×2



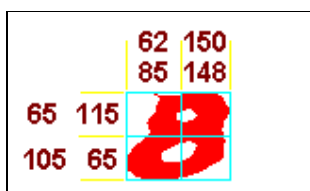
รูปที่ 3-13 สโตรปขนาด 2×2

หลังจากนั้นจึงคำนวณจำนวนครั้งของการทริกเกอร์ทั้งสองแกนของแต่ละช่อง เมื่อค่าสีเปลี่ยนจาก “0” ไปเป็น “1” หรือจากสีขาวเป็นสีดำ ค่าสโตรปทั้งสองแกนของแต่ละช่องจะเป็นค่าเฉลี่ยของแต่ละตัวอักษรที่นำมาทำสโตรป



รูปที่ 3-14 การแบ่งช่องของสโตรป

หลังจากคำนวณได้จำนวนสโตรปทั้งสองแกนของทุกช่องแล้วเมื่อนำไปเทียบกับตัวอักษรต้นแบบ จะมีรูปแบบที่คล้ายคลึงกัน



รูปที่ 3-15 ค่าสโตรปทั้งแกนตั้งและแนวนอนของทุกช่อง